

Publication plan: a self-learning, self-healing mist controller on a 5-dollar chip

Date: 2026-09-24. Builds on [self-healing-gap-check.md](#) and [tinymt-esp32-gap-check.md](#). Evidence: 3 Consensus Deep Searches (listed in section 3), 23 standard Consensus searches and about 30 OpenAlex queries.

1. Verdict: the gap is confirmed

Three independent Deep Searches, each covering 50 to 100 papers, name our study as an open problem without being asked about it:

- **Predictive humidity control on cheap hardware.** The search found that low-cost chips drive ultrasonic humidifiers only with on/off, staged or PID logic. It found no head-to-head experiment of a predictive or learning controller on a cheap microcontroller against an identical on/off baseline. Its first open question is whether approximate predictive control on an Arduino, STM32 or ESP32 can beat threshold on/off control in settling time, overshoot and energy.
- **Learning on the chip.** On-device learning for environmental control exists for solar forecasting, irrigation, lighting and water level. None of it covers humidity or misting. Its gap table marks greenhouses as having no federated learning, and long-running adaptive control on real microcontrollers as sparse.
- **Faults versus normal change.** The search found no papers on actuator faults detected through cross-signal consistency, no papers on residual-based detection deployed at the edge, and no labelled benchmark in which normal drift, sensor faults and actuator faults happen together in a closed-loop IoT system. One of its open questions is when an embedded monitor should keep adapting and when it should freeze and raise a fault.

Our experiment answers one open question from each search. That is the strongest novelty position we have had.

2. The paper

Working title: A self-learning, self-healing humidity controller on a low-cost microcontroller: predictive misting, fault attribution and recovery, validated against on/off control with physically injected faults.

Three contributions

1. **Predictive misting on an ESP32.** A humidity model learned and updated on the chip chooses each mist burst. It is compared head-to-head with the on/off hysteresis controller on the same enclosure.
2. **Fault versus drift attribution on the chip.** The same model's prediction error, plus command-response checks and a consistency check between the two soil probes, separates faults from slow normal change. It then recovers by controlling on its own estimate, switching to the backup mister, or going safe.
3. **An open, labelled dataset.** Normal operation, slow drift and physically injected sensor and actuator faults, all with exact onset times, from a closed-loop misting enclosure.

Claims we can make: first head-to-head experimental comparison of a learning-based predictive controller and on/off control for an ultrasonic mister on a low-cost microcontroller. First on-chip fault attribution and recovery for a plant misting enclosure. First labelled closed-loop dataset with actuator faults for this class of system.

Claims we must not make: first on-device learning (Pinal 2026 did it for solar forecasting), first fault detection in greenhouses (Linker 2000), first virtual humidity sensor (Shekarian 2024, Ponce 2024), or that the method generalises beyond one enclosure.

3. Evidence base (open each paper and record its DOI before citing)

Deep Search	Link	Anchor papers it surfaced
On-device training for environmental control	consensus.app	Pinal 2026 <i>Electronics</i> (ESP32 learns on-device for 115 days, beats frozen copy); Dakhia 2025 <i>Electronics</i> (federated irrigation on STM32); Salem 2025 (Q-learning lighting on an ESP32-class chip); Prasad 2026 (self-calibrating soil probe, drift 5.3 to 1.6 %); CusiHuallpa-Huamantupa 2025 (learned water-level control on Arduino Uno vs PID)
Predictive humidity control on cheap hardware	consensus.app	Güler 2002 <i>J. Med. Eng. Technol.</i> (PIC + ultrasonic nebuliser incubator); Sakib 2021 (Arduino on/off); Deng 2021 (anticipatory switching, ± 2 % RH); Zhang 2025 (STM32 enhanced PID, overshoot 4.3 % vs 23.7 %); Xue 2023 (Raspberry Pi prediction on top of threshold control); Yang 2018 <i>Energy and Buildings</i> (humidity-aware MPC, up to 19.4 % energy saving vs on/off); Drgoňa 2018 (approximate MPC for low-level hardware)
Faults versus normal drift	consensus.app	Kermenov 2023 <i>Sensors</i> (separate drift and anomaly pipelines on a robot); Ramírez 2024 <i>MSSP</i> (faults as input-output model drift); Lee 2024 <i>IJPR</i> (sensor vs process anomaly); Alippi 2017 (sensor fault vs change in the phenomenon); Mavromatis 2022 (drift confirmed by neighbouring devices)

Plus the neighbours already listed in the two earlier gap checks: Linker 2000 and 2002, Munser 2026, Shekarian 2024, Hameed 2017, Ponce 2024, Codeluppi 2021, Ihoume 2022, Sukowati 2026, TinyOL 2021, Disabato 2021 and Bicski 2023.

4. What the study needs: checklist

Research questions

- **RQ1.** Does predictive misting with an on-chip learned model keep humidity in band better than on/off hysteresis, with fewer switch-ons and less water?
- **RQ2.** Can the same model, plus consistency checks, separate injected faults from slow normal drift on the chip, and how fast and accurately?
- **RQ3.** When a fault happens, how much climate damage does on-chip recovery prevent compared with the current rule-based fail-safes?

Controllers compared

Label	What it is
A. On/off	Current firmware: hysteresis 75 to 90 % RH, 5-minute cap, 3-minute cooldown
B. Predictive, frozen	Model fitted on the laptop from days 1 to 3, never updated
C. Predictive, learning	Same model, updated on the chip after every reading

B isolates the value of on-chip learning, the same design as Pinal 2026.

Metrics

Time in the humidity band. Overshoot above the upper limit. Mist switch-ons per day. Mist minutes, which stand for water used. Model prediction error over time. For faults: detection delay, correct attribution rate, false alarms per day on normal data, and time in band during the fault.

Statistics

- **Controllers:** A, B and C run on alternating days in a balanced order. Report day-level means with bootstrap 95 % confidence intervals, and a paired test across days (Wilcoxon signed-rank).
- **Faults:** 7 fault types, at least 5 repeats each, per handling mode. Report every run, plus medians.
- **Thresholds:** fault thresholds are set from normal data only, before any fault is injected. Say so explicitly in the paper, because it prevents tuning to the test.

Hardware

Item	Status
------	--------

ESP32, BME280, 2 soil probes, float, leak sensor, exhaust fan, two mist modules	Have
Reference humidity sensor (SHT31 or second BME280)	Buy, about 500 to 700 taka. Scores the model and is never used by the controller
Salt jar for calibration	Kitchen salt in a sealed jar with a little water holds about 75 % RH at room temperature. Check both humidity sensors in it before starting and report the offsets
Spare mist disc	Buy. Fit a fresh one at the start
Check: do both mist discs spray into the same air?	If not, the backup-mister recovery is dropped and the other recoveries remain

Firmware (to write)

1. **Fast logging:** every 5 s, all sensors, all outputs, the controller label and the model's prediction, as CSV over USB.
2. **Fault marker command:** typing `MARK <fault> START` or `MARK <fault> END` on the serial port stamps the exact onset in the log. This makes the dataset's labels trustworthy, which is the thing the third Deep Search says is missing.
3. **Controllers A, B and C,** selectable by a serial command so days can be switched without reflashing.
4. **The model:** a linear model of the next humidity reading (about 6 numbers), updated by recursive least squares, with a forgetting factor so it follows slow change.
5. **The monitor:** prediction error over a sliding window, command-response check (mist on and no rise), soil-probe agreement check, and a rule for when to adapt versus freeze and alarm.
6. **Recovery actions:** virtual humidity sensor, backup mister, safe mode plus buzzer.

Laptop software (to write)

A logger that saves the serial CSV with timestamps, a fitting script for model B, an analysis notebook that produces every figure and table, and a script that packages the dataset.

Data and code release

Publish the dataset and code on Zenodo, which gives them a DOI, and put the code on GitHub. Include a README with the column list, fault labels, the calibration offsets and the enclosure drawings from 3d/acrylic.

Submission requirements (Computers and Electronics in Agriculture, Elsevier)

- **Highlights:** 3 to 5 bullets, each under 85 characters.
- **Graphical abstract:** optional, recommended.
- **Author statements:** a CRediT contribution statement, a declaration of competing interests, and a data availability statement pointing to the Zenodo DOI.
- **AI disclosure:** a declaration of any generative AI used in writing, which Elsevier requires. It must say plainly where AI tools helped.
- **Cover letter:** stating the three open questions the paper answers, citing the gaps above.
- **Cost:** publishing without open access is free. Open access carries a fee, so check the current amount on the journal page before choosing.
- **Before submitting:** agree the author order among the six team members, and decide whether the course teacher is a co-author.

5. The plan

Days	Work	Output
0	Buy the reference sensor and spare disc. Check the two discs share air. Salt-jar calibration of both humidity sensors	Calibration offsets

1	Write the logging, marker and controller-switch firmware. Laptop logger	Firmware v2 flashed
2 to 4	Run controller A (current on/off) around the clock. This is baseline and training data	3 days of labelled normal data
5	Fit model B on the laptop and compare linear against a small neural net. Port the model and the on-chip update. Set fault thresholds from normal data only	Firmware v3
6 to 11	Controllers A, B and C on alternating days, two days each. Do not touch the box except to refill water at fixed times	RQ1 data
12 to 15	Fault campaign: 7 faults, 5 repeats, each under learning-and-recovery and under the old rules. Mark every onset	RQ2 and RQ3 data
16 to 18	Analysis: all figures and tables from one notebook	Results section
19 to 23	Write the paper, highlights, cover letter and dataset README. Internal review by the team	Draft
24	Upload dataset and code to Zenodo, then submit	Submitted

Suggested team roles for six people: hardware and calibration; firmware; laptop logging and dataset; analysis and figures; writing the introduction, related work and discussion; writing the methods and results, plus the submission paperwork. One person owns each row of the plan.

If time is tight: days 6 to 11 can shrink to one day per controller, and the fault repeats can drop to 3. Both weaken the statistics, so state it in the limitations.

6. Risks

Risk	Answer
"Adaptive control is 50 years old"	Say so and cite it. The contribution is the experiment nobody has run, on hardware nobody has used for it, plus fault attribution and open data
Predictive misting only slightly beats on/off	Report the effect size honestly. Fewer switch-ons and less water at equal time in band is still a finding, and the Deep Search itself frames "is it worth the complexity" as the open question
The neural net does not beat the linear model	Report it. Bicski 2023 is the citation that makes this a result, not a failure
One enclosure, a few weeks	Say it in the limitations. The open dataset and drawings let others repeat it
Reviewer finds a close paper we missed	The three Deep Search reference lists are the related-work backbone. Read the anchor papers in full before writing

7. Side note: JEPA on the ESP32

What JEPA is, simply. A normal predictor guesses the next raw numbers. A JEPA (joint-embedding predictive architecture) first turns a window of readings into a short summary vector, then learns to predict the *summary* of the next window instead of the raw numbers. It ignores noise it cannot predict and focuses on the structure. It learns without labels, which suits fault detection, because faults are rare and unlabelled.

Can it run on the ESP32? Yes, for inference; training stays on the laptop. - An encoder takes the last 60 s of all sensors and actuator states and outputs a 16-number summary. A predictor takes that summary plus the planned mist and fan commands and predicts the next summary. Both are small networks of a few thousand weights. In 8-bit form they need under about 20 KB and run in milliseconds on the ESP32. - Training needs a second copy of the encoder that is slowly averaged, backpropagation, and a regulariser to stop the summaries collapsing. That is practical on the laptop with PyTorch using our logs, but not on the chip. At most, the last layer of the predictor could be updated on the chip. - The fault score is how far the predicted summary is from the actual one. Because the predictor takes the mist and fan commands as inputs, it is an action-conditioned world model of the box.

What already exists - Time-series JEPA for predictive control over a network, tested in simulation on a

cart-pole ([Girgis 2024, arXiv](#)). - JEPA for bearing anomaly detection and remaining life ([Zhang 2026, Advanced Manufacturing](#)). - JEPA for anomaly detection in radar image time series ([Woldesenbet 2026, arXiv](#)). - JEPA for tabular data ([Thimonier 2024, arXiv](#)). - JEPA world models for action planning ([Destrade 2025, arXiv](#)). - The field is growing fast: a broad OpenAlex count of JEPA time-series papers rose from about 5 a year in 2022 to 43 in 2025 and 103 in 2026.

The gap: OpenAlex returned nothing for JEPA on a microcontroller, nothing for JEPA in agriculture or greenhouses, and nothing for a JEPA world model of a physical plant enclosure used for fault detection. This is a lexical search only. Run a Deep Search before claiming "first" (suggested question below).

Honest assessment: with six slow, low-dimensional signals, a JEPA will probably not beat the simple linear model, and a reviewer will ask why it is needed. JEPA earns its place when the input is high-dimensional, for example an ESP32-CAM image of the plant plus the sensors. Then predicting raw pixels is wasteful and predicting summaries makes sense.

Recommendation: keep it out of the main paper's critical path. Either add it as a fourth fault-detection arm if time allows (linear, small neural net, tiny JEPA), or make it a follow-up paper: "TinyJEPA: an action-conditioned world model on a microcontroller for multimodal plant-enclosure monitoring", adding an ESP32-CAM. Target the IEEE Internet of Things Journal or Sensors. The logs from this study are its training data, so nothing is wasted.

Deep Search question to verify the JEPA gap: Has a joint-embedding predictive architecture or other self-supervised world model been deployed on a microcontroller, or used for anomaly detection in agricultural or environmental control systems?
